

ArqDesk

Script de Vídeo — POC I · UFMG

João Correia Costa · Orientador: Prof. José Lopes de Siqueira Neto

Leia em voz alta, no ritmo de uma conversa. Cada parágrafo é uma fala com pausa natural no final. Duração estimada: 5 minutos · ~35 segundos por slide.

SLIDE 1 · TÍTULO

Olá, sou João Costa, esta é uma breve apresentação do POC I. Meu orientador é o Professor José de Siqueira.

O projeto se chama ArqDesk — uma plataforma web de gestão operacional desenvolvida especificamente para escritórios de arquitetura.

SLIDE 2 · INTRODUÇÃO — O PROBLEMA

Para entender o ArqDesk, é preciso entender o ambiente onde ele opera.

Escritórios de arquitetura de pequeno e médio porte gerenciam sua operação de forma fragmentada: projetos em planilhas, comunicação por aplicativos de mensagem sem estrutura, documentos espalhados entre e-mails, e controle de horas feito manualmente. Essa fragmentação tem consequências diretas — atrasos, perda de documentos, e impossibilidade de mensurar o custo real de mão de obra.

As soluções existentes no mercado não resolvem esse problema para esse público. Ferramentas genéricas que carecem de funcionalidades específicas do domínio de arquitetura. Soluções especializadas que focam em construtoras e obras, não em escritórios de projeto. O ArqDesk nasce para preencher essa lacuna.

SLIDE 3 · PROPOSTA

[Objetivo + Metodologia + Diagrama de módulos]

O objetivo é desenvolver o núcleo funcional da plataforma cobrindo quatro grandes áreas: gestão de projetos, gestão de equipe, timesheet e dashboard.

A metodologia adotada é iterativa e incremental, com sprints quinzenais, onde cada entrega deixa o sistema em estado funcional e testável.

A plataforma é organizada em seis módulos que se comunicam através de uma única API REST com 34 endpoints, usando FastAPI, PostgreSQL, SQLAlchemy e Python 3.14.

SLIDE 4 · ARQUITETURA MULTI-TENANT

[Diagrama de schemas isolados no PostgreSQL]

A decisão arquitetural mais importante do projeto foi a estratégia de multi-tenancy — como garantir que os dados de um escritório jamais sejam visíveis para outro, numa plataforma onde vários escritórios coexistem na mesma instância do sistema.

A estratégia adotada foi um schema isolado por tenant dentro do mesmo banco PostgreSQL. Cada escritório tem seu próprio schema, com suas próprias cópias de todas as tabelas. O isolamento é garantido pelo próprio motor do banco — não pelo código da aplicação.

O mecanismo de ativação é elegante: a cada requisição autenticada, a sessão executa `SET search_path TO tenant_x, public`. A partir daí, o ORM não sabe que existe multi-tenancy — ele escreve `SELECT * FROM users` e o PostgreSQL resolve automaticamente para o schema correto. Isso é o que a literatura chama de isolamento transparente.

SLIDE 5 · AUTENTICAÇÃO — JWT

[Payload: sub · tenant_slug · role · exp]

A autenticação usa JSON Web Tokens, o padrão definido na RFC 7519, com o algoritmo HMAC-SHA256. A decisão mais relevante aqui não foi o algoritmo, mas o conteúdo do token.

O payload carrega três campos críticos além do tempo de expiração: o identificador do usuário, o slug do tenant, e o papel global daquele usuário no sistema. Isso torna o token self-contained: toda a informação necessária para autorizar a requisição está nele, sem precisar consultar o banco a cada chamada.

SLIDE 6 · CONTROLE DE ACESSO — RBAC

[Tabela RBAC dois níveis + Pipeline de requisição]

O controle de acesso segue o modelo RBAC, formalizado por Sandhu et al. em 1996, em dois níveis ortogonais: o perfil global no escritório — Admin Master, Admin, Colaborador e Estagiário — e o papel por projeto — Responsável, Co-responsável, Colaborador e Observador.

Esses dois níveis combinam de forma independente. Um administrador global pode ser apenas observador num projeto específico, o que reflete bem a realidade dos escritórios.

Na prática, cada requisição atravessa um pipeline sequencial: valida o JWT, ativa o schema do tenant, verifica o RBAC, e só então executa a lógica de negócio.

SLIDE 7 · PROGRESSO ATUAL

[O processo: TDD · O resultado: 6 módulos · 109 testes · 0 falhas]

O desenvolvimento seguiu TDD — testes escritos antes da implementação em todas as etapas. Isso força um design melhor: as interfaces são definidas pelos casos de uso, não pela conveniência do código.

Em resultado: todos os seis módulos implementados, 109 testes automatizados cobrindo todos eles, execução em dois segundos, zero falhas.

SLIDE 8 · DIFICULDADES ENCONTRADAS

[Isolamento por tenant · Testes com múltiplas camadas · Cobertura limitada a mocks]

A principal dificuldade foi garantir o isolamento de ambiente por tenant em contexto assíncrono — o ciclo de vida das sessões conflitava com o redirecionamento de schema de formas não óbvias.

A segunda foi simular o pipeline completo de segurança de forma verdadeiramente isolada por teste. E a limitação honesta: os 109 testes validam comportamento via mocks, mas nenhuma query foi executada contra o PostgreSQL real.

SLIDE 9 · PRÓXIMOS PASSOS

[Testes de integração · Refresh token · Relatórios financeiros · App mobile]

O mais urgente são os testes de integração com banco real — PostgreSQL em Docker, provisionamento do schema, cenários end-to-end.

Em seguida: refresh token para renovação silenciosa do JWT, relatórios financeiros cruzando horas com taxa horária, e como extensão futura uma aplicação mobile consumindo a API.

SLIDE 10 · CONCLUSÃO

[Quatro pilares · 34 endpoints · 109 testes · 6 migrações · 0 falhas]

Para fechar, o ArqDesk demonstra a aplicação de conceitos centrais de engenharia de software em um problema real.

Multi-tenancy com isolamento garantido pelo banco. API REST tipada com documentação OpenAPI gerada automaticamente. TDD onde as interfaces foram definidas pelos casos de uso. E injeção de dependências para autenticação, roteamento de tenant e controle de acesso sem duplicação de código.

34 endpoints REST. 109 testes. 6 migrações. Zero falhas. Obrigado.